UNIDAD 2. CLASES: LA ESTRUCTURA DEL CÓDIGO

Contenido

1. Clases: La estructura del código 2
   1.1 Concepto de clase 2
   1.2 Estructura de una clase en Java 3
   1.3 Atributos (Campos o Propiedades) 4
   1.4 Métodos (Funciones miembro) 6
   1.5 Constructores 8
2 Objetos: Dando vida a las clases 9
   2.1 Instanciación 9
   2.2 Referencias a objetos 11
   2.3 Componentes del objeto: Identidad, Estado y Comportamiento 13
   2.4 Mensajes (Paso de mensajes) 14
3 Encapsulado y visibilidad 16
   3.1 El principio de ocultación: concepto de "caja negra" 16
   3.2 Modificadores de acceso en Java 17
   3.3 Métodos de acceso (Getters y Setters) 18
   3.4 Inmutabilidad 20
4 Relaciones básicas entre clases 22
   4.1 Dependencia (Uso) 22
   4.2 Asociación 24
   4.3 Agregación y Composición 26
   4.4 Introducción a la Herencia (Generalización) 28
5 Principios básicos de la orientación a objetos 30
   5.1 Abstracción: Simplificación de la realidad 30
   5.2 Encapsulamiento 31
   5.3 Herencia: Reutilizar código 32
   5.4 Polimorfismo: Flexibilidad de comportamiento 34
Resumen 36

1. Clases: La estructura del código

1.1 Concepto de clase

En la Programación Orientada a Objetos (POO), una **Clase** es una **plantilla, molde o prototipo** que define la forma y el comportamiento de una entidad.

No es la cosa real, sino la **definición abstracta** de esa cosa.

La mejor analogía: El plano y la casa

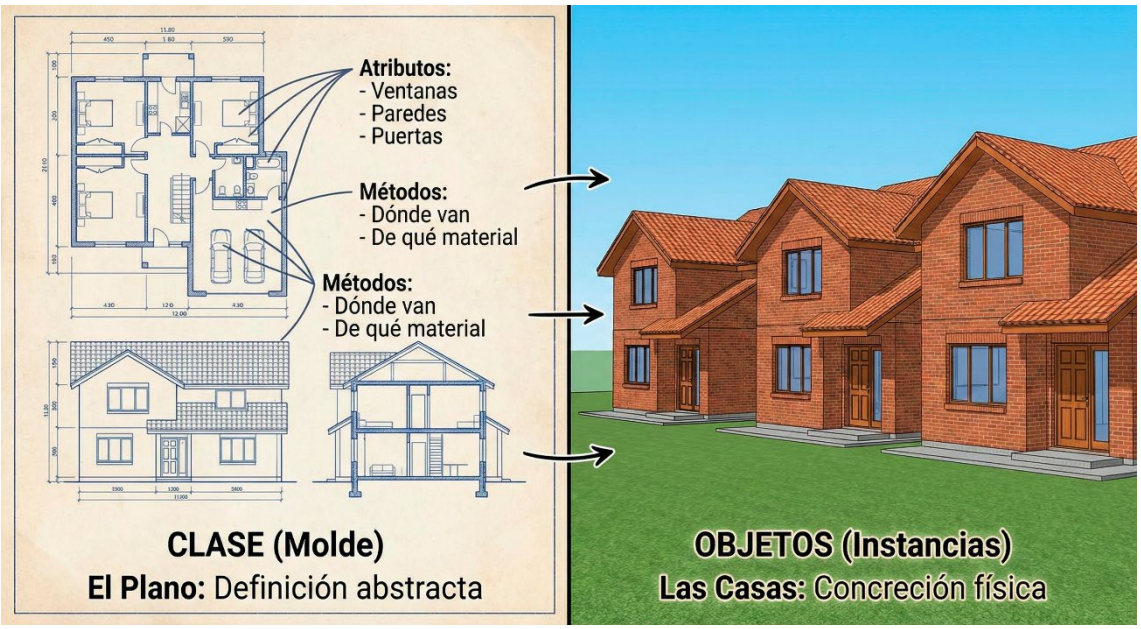
Para entenderlo, piensa en un arquitecto:

1. **La Clase (El Plano):** El arquitecto dibuja un plano. En ese papel dice dónde van las ventanas, de qué material son las paredes y cuántas puertas hay.

   ¿Puedes vivir en el plano? No. Es solo papel. Es una definición.

2. **El Objeto (La Casa):** El constructor toma ese plano y construye una casa real con ladrillos. Luego usa el *mismo* plano para construir otra casa idéntica al lado.

   ¿Puedes vivir en la casa? Sí. Es la concreción física del plano.



¿Qué define una clase?

Una clase agrupa dos cosas inseparables que describen a la entidad:

1. **Atributos (Datos):** Describen **cómo es**. Son las características o propiedades.

   Ejemplo para una clase `Coche`*:* Color, marca, velocidad actual, cantidad de gasolina.

2. **Métodos (Comportamiento):** Describen **qué puede hacer**. Son las acciones o funciones.

   Ejemplo para una clase `Coche`*:* Arrancar, frenar, girar.

Abstracción: El arte de simplificar

Cuando diseñamos una clase, usamos la **abstracción**. Esto significa que no modelamos *todo* lo que tiene un coche real (no nos importa la marca de los tornillos del motor), sino solo **lo que es relevante para nuestro programa**.

- Si hacemos un sistema de multas de tráfico, la clase `Coche` tendrá el atributo `matricula`.
- Si hacemos un videojuego de carreras, la clase `Coche` tendrá el atributo `velocidadMaxima`.

En el contexto de Java

En Java, todo el código debe vivir dentro de una clase. Es la unidad organizativa básica.

- Se define con la palabra reservada `class`.
- Por convención, los nombres de las clases siempre empiezan con **Mayúscula** (PascalCase). Ejemplo: `Coche`, `CuentaBancaria`, `Usuario`.

1.2 Estructura de una clase en Java

Para crear una clase en Java, necesitamos definir un archivo de texto con extensión `.java` que siga una sintaxis muy estricta.

La Sintaxis Básica

Una clase mínima se define con la palabra reservada `class`, seguida de un nombre y un bloque de código delimitado por llaves `{ }`.

Java
```
[Modificador] class NombreDeLaClase {
    // 1. Aquí se declaran los ATRIBUTOS (Variables)

    // 2. Aquí se declaran los MÉTODOS (Funciones)
}
```

Elementos paso a paso:

1. **Modificador de acceso** (`public`): Generalmente verás public class. Esto significa que esta clase puede ser utilizada por cualquier otra parte del código.
2. **Palabra reservada** (`class`): Le dice al compilador: "Oye, lo que viene ahora es la definición de un nuevo tipo de objeto".
3. **Nombre de la clase** (`Coche`): Es el identificador único.

   - **Regla de oro:** En Java, los nombres de las clases siempre usan **PascalCase** (la primera letra de cada palabra en mayúscula).

- *Bien:* `Coche`, `CuentaBancaria`, `UsuarioRegistrado`.
- *Mal:* `coche`, `cuenta_bancaria`, `USUARIO`.

4. **El Cuerpo** (`{ ... }`): Todo lo que esté entre la llave de apertura { y la de cierre } pertenece a la clase. Fuera de las llaves no puede haber código lógico.

La Regla del Archivo (Muy Importante)

En Java existe una regla física vinculada al sistema de archivos:

> Si tienes una clase pública llamada `Coche`, el archivo **obligatoriamente** debe llamarse `Coche.java`.
>
> Si llamas al archivo `coche.java` (minúscula) o `Vehiculo.java`, el programa **no compilará**.

Ejemplo Real

Vamos a traducir el plano del arquitecto a código Java real:

Java
```
// Archivo: Coche.java

public class Coche {

    // --- ZONA DE ATRIBUTOS (Estado) ---
    // Aquí definimos las variables que tendrá cada coche
    String marca;
    String color;
    int velocidad;

    // --- ZONA DE MÉTODOS (Comportamiento) ---
    // Aquí definimos las funciones
    void arrancar() {
        // Lógica para arrancar
    }

    void frenar() {
        // Lógica para frenar
    }
}
```

Más info: Aunque Java permite mezclar atributos y métodos, el estándar de la industria es declarar **primero todos los atributos** arriba del todo y **después todos los métodos**.

1.3 Atributos (Campos o Propiedades)

Los **atributos** (también llamados campos, propiedades o variables de instancia) son las variables que se declaran dentro de la clase, pero **fuera** de cualquier método.

Representan el **estado** del objeto. Si la clase es el sustantivo, los atributos son sus adjetivos o características.

Sintaxis de Declaración

Se definen igual que una variable normal en Java, indicando primero el tipo de dato y luego el nombre.

Java
```
[Visibilidad] TipoDeDato nombreDelAtributo [= ValorInicial];
```

- **Visibilidad:** (Opcional por ahora, veremos `public`/`private` en el punto 2.3).
- **Tipo de Dato:** Puede ser un tipo primitivo (`int`, `double`, `boolean`) o una clase (`String`, `Date`).
- **Nombre:** Debe seguir la convención **camelCase** (empezar en minúscula).

Ejemplo Práctico

Siguiendo con nuestro `Coche`, definamos sus atributos:

Java
```
public class Coche {
    // Atributos de tipo Objeto (Clases)
    String marca;
    String modelo;
    String color;

    // Atributos de tipo Primitivo
    int velocidadActual;
    boolean motorEncendido;
    double precio;
}
```

Concepto Clave: Variables de Instancia

A estos atributos se les llama **variables de instancia**. ¿Por qué? Porque cada objeto (instancia) que creemos tendrá **su propia copia** de estas variables.

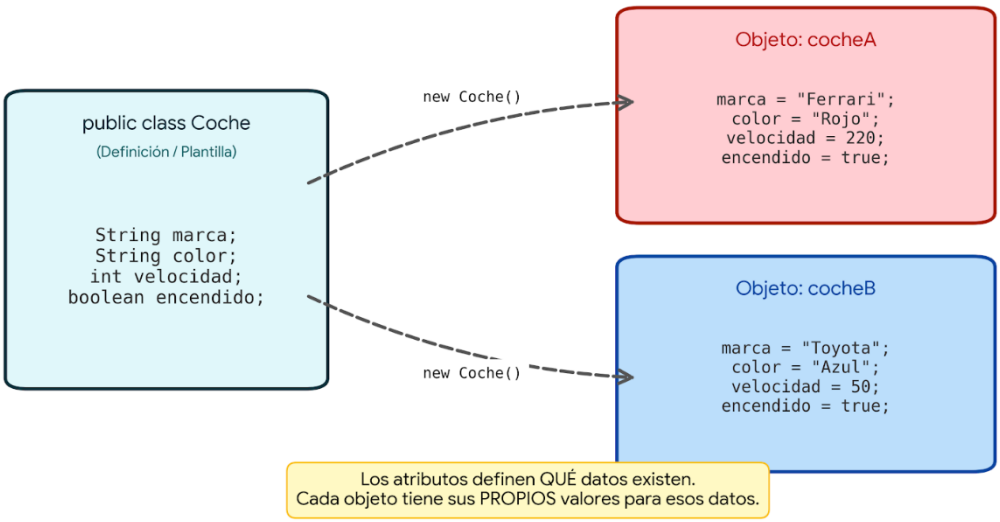
- Si creo el `Coche A`, puede tener `color = "Rojo"`.
- Si creo el `Coche B`, puede tener `color = "Azul"`.
- Modificar la velocidad del `Coche A` **no afecta** a la velocidad del `Coche B`. Son independientes.

Valores por Defecto (Inicialización Automática)

A diferencia de las variables que creas dentro de un método (variables locales), si no le das un valor inicial a un atributo, Java le asigna uno por defecto automáticamente al crear el objeto:

Tipo de Dato	Valor por Defecto
`int`, `byte`, `short`, `long`	`0`
`double`, `float`	`0.0`

Tipo de Dato	Valor por Defecto
`boolean`	`false`
Objetos (String, Coche, etc.)	`null` (Vacío/Nulo)



Importante: Aunque Java los inicializa por ti, es una buena práctica (Clean Code) no depender de estos valores por defecto si la lógica de tu programa requiere un valor específico inicial (ej. que la `velocidadActual` empiece explícitamente en 0).

1.4 Métodos (Funciones miembro)

Un **método** es un bloque de código que contiene una serie de instrucciones programadas para realizar una tarea específica. En el paradigma Orientado a Objetos, los métodos representan el **comportamiento** de la entidad.

Si pensamos gramaticalmente:

- **Clase/Objeto** = Sustantivo (*Coche*)
- **Atributos** = Adjetivos (*Rojo, Rápido*)
- **Métodos** = Verbos (*Arrancar, Frenar, Girar*)

La sintaxis en Java para definir un método es la siguiente:

Java
```
[Modificador] TipoDeRetorno nombreDelMetodo(Tipo Parametro1, Tipo
Parametro2) {
    // CUERPO DEL MÉTODO
    // Instrucciones...
    return valor; // (Opcional, depende del tipo de retorno)
}
```

Vamos a desglosar las partes clave:

1. **Tipo de Retorno (Output):** Indica qué tipo de dato devuelve el método al terminar su trabajo.

    - Si el método hace un cálculo y devuelve un número, el tipo será `int` o `double`.
    - `void` **(Vacío):** Es una palabra clave fundamental. Significa que el método realiza una acción pero **no devuelve ningún valor**. (Ej: Un método `imprimir()` solo muestra texto, no devuelve un dato para seguir operando).

2. **Nombre del Método:** Sigue la convención camelCase (empieza en minúscula). Debe ser un verbo o una frase verbal (`calcularPrecio, abrirPuerta`).
3. **Parámetros (Input):** Son las variables que el método necesita recibir para funcionar. Van entre paréntesis. Si no necesita nada, los paréntesis se dejan vacíos ().
4. **Cuerpo** ({ ... }): El algoritmo que resuelve la tarea.

Ejemplo Práctico en la clase `Coche`

Vamos a añadir comportamiento a nuestro coche. Queremos que pueda acelerar (aumentando su velocidad) y que nos diga cuánta gasolina le queda.

Java
```
public class Coche {
    // Atributos (Estado)
    int velocidadActual = 0;
    int gasolina = 50;

    // --- MÉTODOS (Comportamiento) ---

    // 1. Método VOID: Hace algo, pero no devuelve nada.
    // Recibe un parámetro 'cantidad' para saber cuánto acelerar.
    void acelerar(int cantidad) {
        if (gasolina > 0) {
            velocidadActual = velocidadActual + cantidad;
            gasolina = gasolina - 1; // Gasta un poco de gasolina
            System.out.println("El coche ha acelerado. Velocidad: "
+ velocidadActual);
        } else {
            System.out.println("No queda gasolina.");
        }
    }

    // 2. Método con RETORNO: Calcula y devuelve un dato.
    // No recibe parámetros ().
    int obtenerGasolina() {
        return gasolina; // 'return' es obligatorio aquí porque
dijimos 'int' arriba
    }
}
```

La importancia del `return`

Es vital distinguir entre **imprimir** un valor y **retornar** un valor.

- `System.out.println(gasolina)`: Muestra el dato en la pantalla negra para que lo vea el humano, pero el programa "pierde" el dato.

- `return gasolina`: Entrega el dato a otra parte del programa para que siga haciendo cálculos con él.

Analogía:

- `void`: Le pides al cocinero que tire la basura. Lo hace y vuelve sin nada.
- `return`: Le pides al cocinero una hamburguesa. Él trabaja y te **devuelve** la hamburguesa en la mano.

1.5 Constructores

Un **Constructor** es un bloque de código especial dentro de la clase que se ejecuta **automáticamente** en el momento en que se crea una nueva instancia (objeto) de esa clase.

Su misión principal es **inicializar el objeto**: dar valores iniciales a los atributos para que el objeto no esté vacío o en un estado inválido al nacer.

Reglas de Oro (Sintaxis)

Para que Java reconozca que un método es en realidad un constructor, debe cumplir dos reglas estrictas:

1. **El nombre debe ser idéntico al de la clase.** (Si la clase es `Coche`, el constructor se llama `Coche`).
2. **No tiene tipo de retorno.** No se pone `void`, ni `int`, ni nada. Si pones `void Coche()`, Java pensará que es un método normal y no funcionará como constructor.

Tipos de Constructores

1. El Constructor por Defecto (Vacío): Si tú no escribes ningún constructor en tu clase, Java crea uno invisible por ti que no hace nada (solo inicializa variables a 0 o null).

- *Uso:* `new Coche();`

2. El Constructor Parametrizado (Con argumentos): Es el que escribimos nosotros para obligar a quien cree el objeto a darle datos iniciales.

Java
```
public class Coche {
    String marca;
    String color;

    // ESTE ES EL CONSTRUCTOR
    // Se ejecuta al hacer 'new Coche("Ford", "Rojo")'
    public Coche(String marcaRecibida, String colorRecibido) {
        marca = marcaRecibida;
        color = colorRecibido;
    }
}
```

Importante: En cuanto escribes *cualquier* constructor manualmente, Java deja de regalarte el constructor vacío por defecto. Si quieres mantener ambos, debes escribir ambos (esto se llama **Sobrecarga de constructores**).

La palabra reservada `this`

Es muy común que el parámetro del constructor se llame igual que el atributo de la clase (por claridad). Para que Java no se confunda, usamos `this`.

- **`this`** significa: "Este objeto actual". Se usa para referirse a los atributos de la propia clase, diferenciándolos de los parámetros que llegan de fuera.

Java
```
public class Coche {
    String marca; // Atributo de la clase

    public Coche(String marca) { // Parámetro del constructor
        // marca = marca; // ¡ERROR! Ambigüedad. Java no sabe cuál
es cuál.

        this.marca = marca;
        // LÉASE: "A la marca DE ESTE OBJETO (this), asígnale la
marca que llega por PARÁMETRO"
    }
}
```

Diferencias con un método normal

Característica	Constructor	Método Normal
Nombre	Idéntico a la Clase	Verbo o frase descriptiva
Retorno	Ninguno (ni siquiera `void`)	Obligatorio (`void`, `int`, etc.)
Cuándo se llama	Automáticamente al hacer `new`	Manualmente (`objeto.metodo()`)
Propósito	Inicializar	Realizar acciones/lógica

2 Objetos: Dando vida a las clases

2.1 Instanciación

La **instanciación** es el proceso de crear un objeto concreto a partir de una clase. En la terminología de Java, a un objeto también se le llama **instancia**.

Si la clase es el molde de galletas, la instanciación es el acto de presionar el molde sobre la masa para crear una galleta real.

El operador `new`

En Java, la palabra mágica para crear objetos es **`new`**. Este operador realiza tres tareas vitales en una sola línea:

1. **Reserva memoria:** Busca espacio libre en la memoria RAM del ordenador para guardar los datos del nuevo objeto.
2. **Crea el objeto:** Instancia la estructura definida en la clase.
3. **Llama al constructor:** Ejecuta el código de inicialización que vimos en el punto anterior.

Sintaxis y Desglose

La línea de código fundamental que escribirás miles de veces es esta:

Java
```
NombreClase nombreVariable = new NombreClase(argumentos);
```

Vamos a diseccionar un ejemplo real usando nuestra clase `Coche`:
Java
```
Coche miFerrari = new Coche("Ferrari", "Rojo");
```

Esta línea simple tiene dos partes muy distintas separadas por el signo igual `=`:

1. **La Declaración (Izquierda):** `Coche miFerrari`
   - Aquí le dices a Java: "Reserva un nombre `miFerrari` que servirá para manejar un objeto de tipo `Coche`".
   - *Ojo:* Aquí todavía no existe el coche real, solo la etiqueta o el "mando a distancia" para manejarlo.
2. **La Instanciación (Derecha):** `new Coche("Ferrari", "Rojo")`
   - Aquí es donde ocurre la magia. `new` crea el objeto en memoria y el Constructor guarda "Ferrari" y "Rojo" dentro de él.
   - Devuelve la dirección de memoria donde ha aparcado ese coche.
3. **La Asignación (=):**
   - Conecta la etiqueta (`miFerrari`) con el objeto real creado en memoria.

Creación de múltiples instancias

La gran potencia de la POO es que puedes crear infinitos objetos independientes usando la misma clase:

Java
```
// Instancia 1
Coche cocheDeJuan = new Coche("Ford", "Azul");

// Instancia 2
Coche cocheDeMaria = new Coche("Toyota", "Blanco");
```

- `cocheDeJuan` tiene su propia velocidad y gasolina.

- `cocheDeMaria` tiene su propia velocidad y gasolina.

Son totalmente independientes. Si Juan choca su coche, el de María sigue intacto.

2.2 Referencias a objetos

En Java, hay una gran diferencia entre los tipos primitivos (como `int` o `boolean`) y los objetos.

- **Variables Primitivas:** Son cajas que contienen el **valor** real.
  - `int edad = 25;` -> La caja `edad` contiene el número 25.
- **Variables de Objeto (Referencias):** Son cajas que contienen la **dirección de memoria** donde vive el objeto. No contienen el objeto en sí.

La Analogía del Mando a Distancia

La mejor forma de entender esto es imaginar que:

1. **El Objeto (creado con `new`)** es un **Televisor** real.
2. **La Variable** no es el televisor, es el **Mando a Distancia**.

Cuando escribes:

Java
```
Coche miCoche = new Coche();
```

- `new Coche()` fabrica el televisor (el objeto) y lo pone en el salón.
- `miCoche` es el mando a distancia que se vincula a ese televisor.

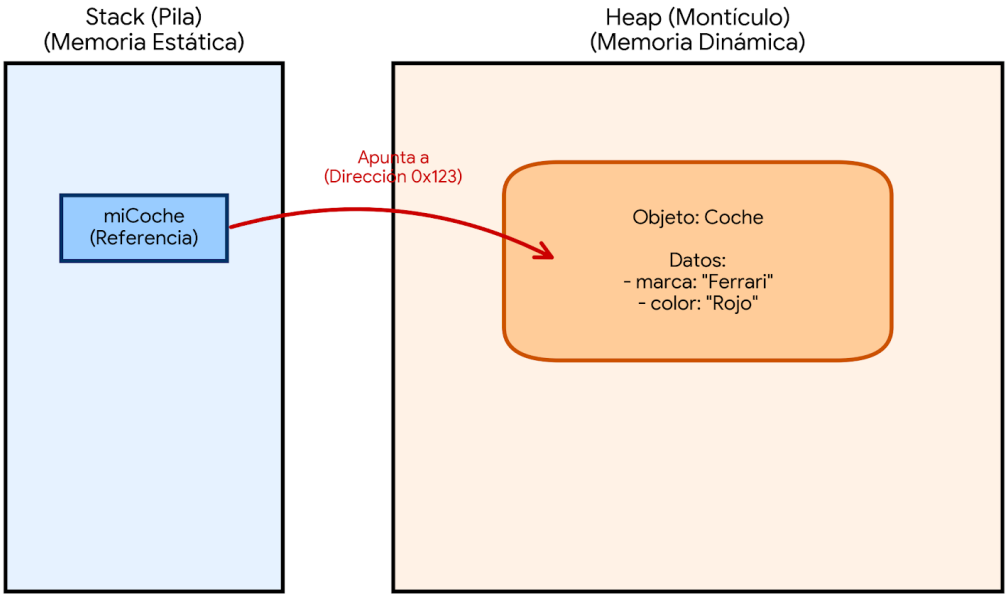
Gestión de Memoria: Stack vs. Heap

Para que esto funcione, Java divide la memoria en dos zonas:

1. **Stack (Pila):** Es una memoria pequeña y rápida donde viven las variables (los mandos a distancia). Aquí vive la variable `miCoche`.
2. **Heap (Montículo):** Es una memoria grande y dinámica donde viven los objetos reales. Aquí vive el objeto `Coche` con sus datos (color, marca, etc.).

La variable en el Stack guarda una "flecha" o dirección (ej: `0x123`) que apunta al objeto en el Heap.

Estructura de Memoria: Stack vs. Heap

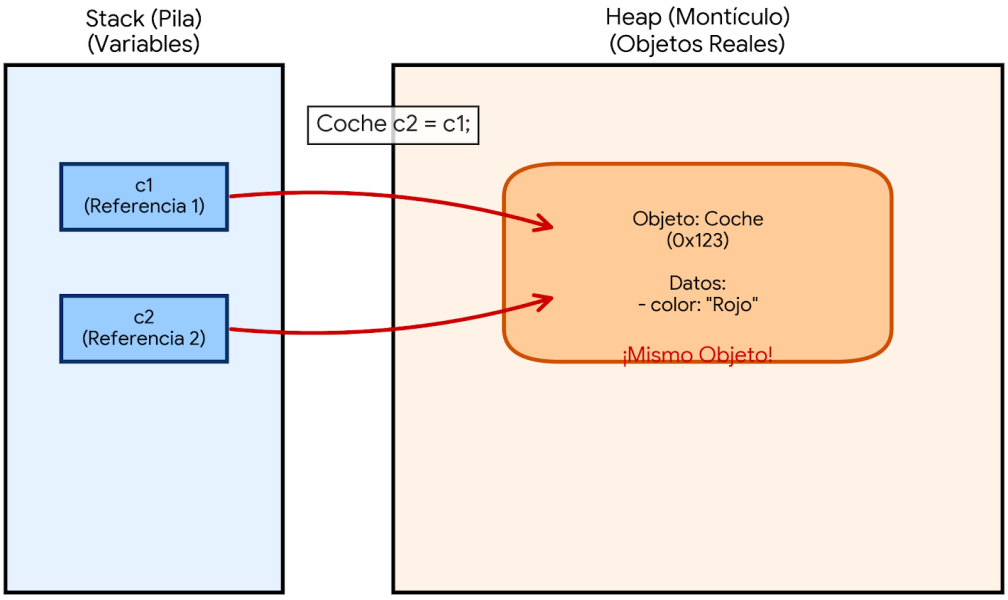


Consecuencia Clave: Copia de Referencias

Esto tiene un impacto enorme en cómo programamos. Mira este ejemplo:

Java
```
Coche c1 = new Coche(); // c1 apunta al Ferrari
c1.color = "Rojo";

Coche c2 = c1; // ¡CUIDADO AQUÍ!
```

Problema de Referencias Compartidas



¿Qué ha pasado en la línea `Coche c2 = c1;`?

- **No** se ha clonado el coche. No tenemos dos Ferraris.
- Se ha copiado la dirección. Ahora tienes **dos mandos a distancia (c1 y c2) apuntando al MISMO coche**.

Si ahora haces:

Java
```
c2.color = "Azul";
System.out.println(c1.color); // Imprimirá "Azul"
```

Como ambos apuntan al mismo objeto, si lo modificas usando `c2`, `c1` ve el cambio. Son "alias" del mismo objeto.

El valor `null`

¿Qué pasa si tienes un mando a distancia pero no está conectado a ningún televisor?

En Java, eso se representa con la palabra clave **`null`**.
Java
```
Coche miCoche = null;
```

Significa que la variable existe, pero **no apunta a ningún objeto** en memoria. Es una referencia vacía.

¡Peligro! NullPointerException Si intentas usar un mando que no apunta a nada, el programa explota.

`miCoche.arrancar();` // Si miCoche es null, esto lanza un error fatal (Excepción) porque no hay coche que arrancar.

2.3 Componentes del objeto: Identidad, Estado y Comportamiento

1. Identidad (Identity)

La identidad es la propiedad que permite distinguir un objeto de otro, independientemente de que tengan los mismos datos. Es lo que hace que un objeto sea **único**.

- **Analogía del mundo real:** Imagina dos monedas de 1 euro recién salidas de fábrica.
    - Tienen el mismo valor, el mismo peso, el mismo dibujo y el mismo año.
    - Sin embargo, son **dos** monedas físicas distintas. Si fundes una, la otra sigue existiendo. Su identidad es su existencia física separada.
- **En Java:** La identidad viene dada por la **dirección de memoria** donde está almacenado el objeto. Aunque dos objetos tengan exactamente los mismos datos dentro, si están en direcciones de memoria diferentes, tienen identidades diferentes.

En Java, el operador == compara identidad (direcciones de memoria), no contenido.

- `cocheA == cocheB` solo será `true` si ambos son *exactamente el mismo objeto* (el mismo mando a distancia apuntando al mismo sitio).

2. Estado (State)

El estado es el conjunto de los valores de todos los atributos de un objeto en un instante determinado. Es la "foto" del objeto en un momento concreto.

- **Dinamismo:** El estado es **mutable** (cambiante). El objeto evoluciona a lo largo del tiempo.
- **Ejemplo:**
    - *Instante 1:* El coche está parado. Estado: `{ velocidad: 0, encendido: false }`.
    - *Instante 2 (después de arrancar):* El coche está al ralentí. Estado: `{ velocidad: 0, encendido: true }`.
    - *Instante 3 (después de acelerar):* El coche se mueve. Estado: `{ velocidad: 50, encendido: true }`.

El estado se almacena en las **variables de instancia** (atributos) que definimos en la clase.

3. Comportamiento (Behavior)

El comportamiento es lo que el objeto **puede hacer** y cómo **responde** ante las peticiones de otros objetos.

- Está definido por los **métodos** de la clase.
- El comportamiento suele actuar sobre el estado:
    - El método `frenar()` (comportamiento) modifica la variable `velocidad` (estado).
    - El método `pintar()` (comportamiento) modifica la variable `color` (estado).

Imagina un objeto `CuentaBancaria`:

Componente	**Qué es**	**En el código (Java)**	**Ejemplo**
Identidad	Su "DNI" único en memoria	La referencia (`@1a2b3c`)	Es *esa* cuenta y no otra idéntica.
Estado	Sus datos actuales	Los valores de sus atributos	Saldo: 1000€, Titular: "Ana".
Comportamiento	Sus capacidades	Sus métodos	`ingresar()`, `retirar()`.

2.4 Mensajes (Paso de mensajes)

Un programa orientado a objetos no es más que un conjunto de objetos colaborando entre sí. Como los objetos están aislados (cada uno tiene su propia memoria y estado), necesitan una forma de comunicarse para trabajar juntos. Esta comunicación se llama **Paso de Mensajes**.

En términos prácticos, en Java, **enviar un mensaje es lo mismo que llamar a un método** de otro objeto.

¿Qué compone un mensaje?

Cuando un objeto A quiere que el objeto B haga algo, le envía un mensaje compuesto por tres partes fundamentales:

1. **El Destinatario (Receptor):** ¿A quién le hablo? (La referencia al objeto).
2. **El Selector (Método):** ¿Qué quiero que haga? (El nombre del método).
3. **Los Argumentos (Parámetros):** ¿Qué información necesita para hacerlo? (Datos adicionales).

Sintaxis en Java: El Operador Punto (.)

El "pegamento" que permite esta comunicación en Java es el **operador punto**.
Java
```
receptor.metodo(argumentos);
```

Ejemplo de la vida real

Imagina una interacción entre un **Conductor** y un **Coche**.

- El Conductor no gira las ruedas con sus manos directamente (eso sería manipular el estado interno).
- El Conductor utiliza el volante. El volante es el mecanismo para enviar el mensaje.

Java
```
// Escenario: El objeto 'juan' (Conductor) interactúa con el objeto
'ferrari' (Coche)

ferrari.arrancar();
// 1. Receptor: ferrari
// 2. Mensaje: arrancar
// 3. Argumentos: (ninguno)

ferrari.acelerar(50);
// 1. Receptor: ferrari
// 2. Mensaje: acelerar
// 3. Argumentos: 50 (la cantidad)
```

Flujo de Control (Sincronía)
Es importante entender qué pasa con el programa cuando se envía un mensaje:

1. El objeto **Emisor** (quien llama) se detiene y espera.
2. El control del programa salta al objeto **Receptor**.
3. El Receptor ejecuta su método (su comportamiento).
4. Si el método devuelve algo (`return`), el Receptor le entrega el valor al Emisor.
5. El **Emisor** recupera el control y continúa su ejecución.

3 Encapsulado y visibilidad

Hasta ahora, nuestros objetos eran "transparentes". Cualquiera podía tocar sus atributos y cambiar sus datos libremente. En el desarrollo profesional, esto es peligroso. El **encapsulamiento** es el mecanismo que permite proteger la información del objeto, decidiendo qué partes son visibles para el exterior y cuáles se mantienen ocultas.

3.1 El principio de ocultación: concepto de "caja negra"

El principio de ocultación de información dicta que **los detalles internos de funcionamiento de un objeto deben permanecer ocultos al mundo exterior**.

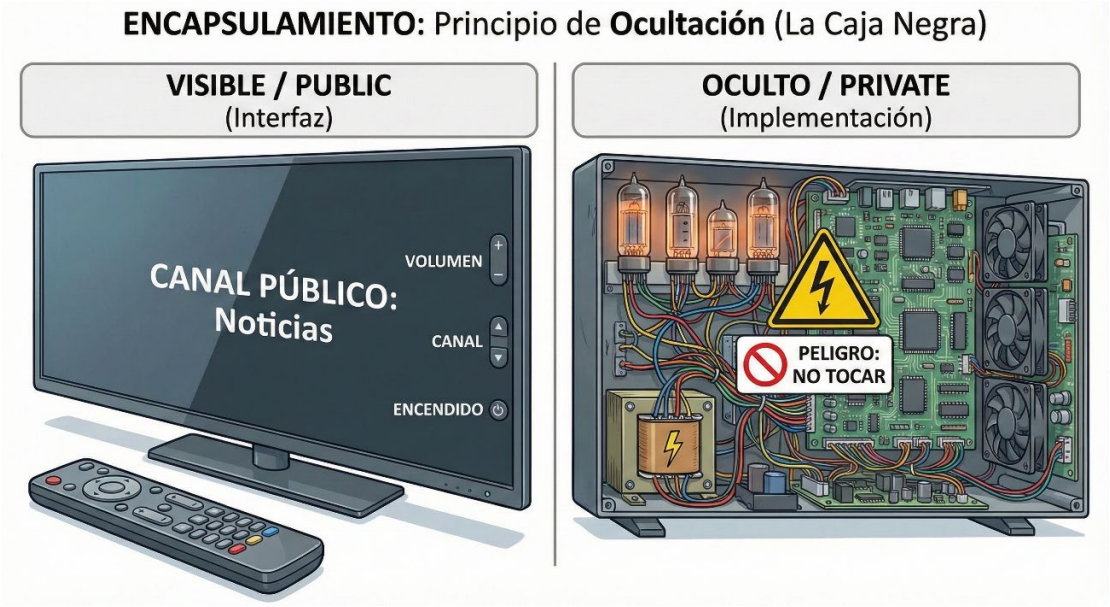
Solo debemos exponer aquello que es estrictamente necesario para usar el objeto (su interfaz), escondiendo la complejidad de cómo hace las cosas por dentro.

La Metáfora de la "Caja Negra"
Para entender este concepto, usamos la analogía de la ingeniería conocida como **Caja Negra**.
Imagina un **Televisor**:

- **Lo que ves (Interfaz Pública):** El mando a distancia, la pantalla y los botones de volumen. Esto es lo que usas para interactuar con él.
- **Lo que NO ves (Implementación Privada):** Los circuitos electrónicos, el transformador de corriente, los cables internos y los chips de procesamiento.

Tú (el usuario u otro objeto) no necesitas saber electrónica para ver las noticias. Es más, el fabricante **cierra** la caja del televisor para protegerlo.

Si pudieras tocar los cables mientras funciona, podrías electrocutarte o romper el televisor. Al ocultar los cables (encapsularlos), el televisor es **seguro** y **fácil de usar**.



¿Por qué aplicamos esto al software?

En programación, hacemos lo mismo. Un objeto debe ser una caja negra:

1. **Protección de la integridad (Seguridad):** Imagina una clase CuentaBancaria. Si el atributo saldo fuera accesible directamente, cualquier parte del código podría hacer: `cuenta.saldo = -5000000;` Esto sería catastrófico. Al ocultar el saldo dentro de la caja negra, obligamos a usar métodos como retirarDinero(), que incluyen lógica de protección ("Si saldo < cantidad, denegar operación").
2. **Independencia (Mantenibilidad):** Si ocultas cómo funciona tu objeto por dentro, puedes cambiar su código interno mañana (para hacerlo más rápido o mejor) sin que el resto del programa se entere ni se rompa. Mientras no cambies los "botones" de fuera, lo de dentro es asunto tuyo.
3. **Reducción de complejidad:** Al ocultar los datos internos, el programador que usa tu clase no tiene que preocuparse por docenas de variables. Solo ve los 3 o 4 métodos necesarios para trabajar.

En resumen, el encapsulamiento dice: "Yo me encargo de mis datos. Tú solo dime qué quieres hacer (envíame un mensaje) y yo decidiré cómo hacerlo y si es seguro hacerlo".

3.2 Modificadores de acceso en Java

Los modificadores de acceso son palabras reservadas que colocamos al principio de la definición de clases, atributos o métodos para configurar su nivel de visibilidad. Determinan **quién** puede ver y usar esas partes del código.

En Java, existen 4 niveles de acceso, ordenados del más restrictivo (más cerrado) al menos restrictivo (más abierto):

1. `private` (Privado) - El más estricto

Es el pilar del encapsulamiento.

- **Visibilidad:** Solo visible dentro de la **misma clase**.
- **Comportamiento:** Nadie desde fuera (ni siquiera otras clases en el mismo archivo) puede acceder a un atributo o método privado. Es "solo para mis ojos".
- **Uso:** Se usa para todos los **atributos** (datos) y para métodos internos auxiliares.

2. (Default / Package-private) - Sin palabra clave

Es lo que ocurre si no escribes nada.

- **Visibilidad:** Visible para cualquier clase que esté dentro del **mismo paquete** (carpeta).
- **Comportamiento:** Es como una "confianza vecinal". Si tu clase `Coche` y tu clase `Taller` están en el mismo paquete, el `Taller` puede ver lo que sea *default* en `Coche`.
- **Uso:** A menudo se evita en favor de ser explícito con `private` o `public`, aunque es útil en diseños modulares específicos.

3. `protected` **(Protegido)**

Este cobra sentido cuando vemos **Herencia** (Unidad 3).

- **Visibilidad:** Visible en el mismo paquete **Y** también en cualquier **subclase** (hija), aunque esté en otro paquete.
- **Comportamiento:** Es una "herencia familiar". Los datos son privados para el mundo, pero accesibles para la familia (clases hijas).

4. `public` **(Público) - El más permisivo**

- **Visibilidad:** Visible para **todo el mundo**. Cualquier clase, en cualquier paquete, puede acceder.
- **Comportamiento:** Es la interfaz pública del objeto.
- **Uso:** Se usa para los **métodos** que queremos que otros usen (`arrancar()`, `getSaldo()`) y para la propia definición de la Clase.

Ejemplo en Código: La regla de oro

La regla estándar en la programación profesional Java (POJO - Plain Old Java Object) es:

"Atributos privados, Métodos públicos".
Java
```
public class CuentaBancaria {

    // 1. ATRIBUTOS PRIVATE (Cerrados con llave)
    // Nadie desde fuera puede hacer: cuenta.saldo = 0;
    private double saldo;
    private int pinSecreto;

    // 2. MÉTODOS PUBLIC (La ventanilla de atención)
    // Todo el mundo puede llamar a estos métodos para interactuar
de forma segura.
    public void ingresarDinero(double cantidad) {
        if (cantidad > 0) {
            saldo = saldo + cantidad; // La propia clase SÍ puede
tocar su atributo privado
        }
    }
}
```

Al hacer el `saldo` privado, evitamos que alguien lo modifique sin pasar por las validaciones del método público `ingresarDinero`.

3.3 Métodos de acceso (Getters y Setters)

Como acabamos de establecer la regla de oro **"Atributos privados"**, nos encontramos con un problema práctico: si hago `private int edad`, ya nadie puede consultar la edad ni modificarla. El objeto se ha vuelto inútil.

Para solucionar esto sin romper el encapsulamiento, creamos métodos públicos específicos que actúan como **intermediarios** o "porteros".

1. El Getter (Método de Acceso / Lectura)

Es un método público cuya única misión es **devolver** el valor de un atributo privado.

- **Convención de nombre:** `get` + NombreDelAtributo (en CamelCase).
- **Excepción:** Si el atributo es `boolean`, se suele usar `is` + NombreDelAtributo (ej: `isEncendido` en lugar de `getEncendido`).

Java
```
// Atributo privado
private String nombre;

// Getter público
public String getNombre() {
    return this.nombre; // Devuelve el valor al exterior
}
```

2. El Setter (Método de Modificación / Escritura)

Es un método público cuya misión es **asignar** un nuevo valor a un atributo privado. Aquí es donde reside la magia de la seguridad, ya que podemos incluir lógica de validación.

- **Convención de nombre:** `set` + NombreDelAtributo.
- **Retorno:** Generalmente es `void` (no devuelve nada).

Java
```
// Setter público
public void setNombre(String nuevoNombre) {
    this.nombre = nuevoNombre; // Asigna el valor
}
```

¿Por qué usarlos en lugar de hacer el atributo público?

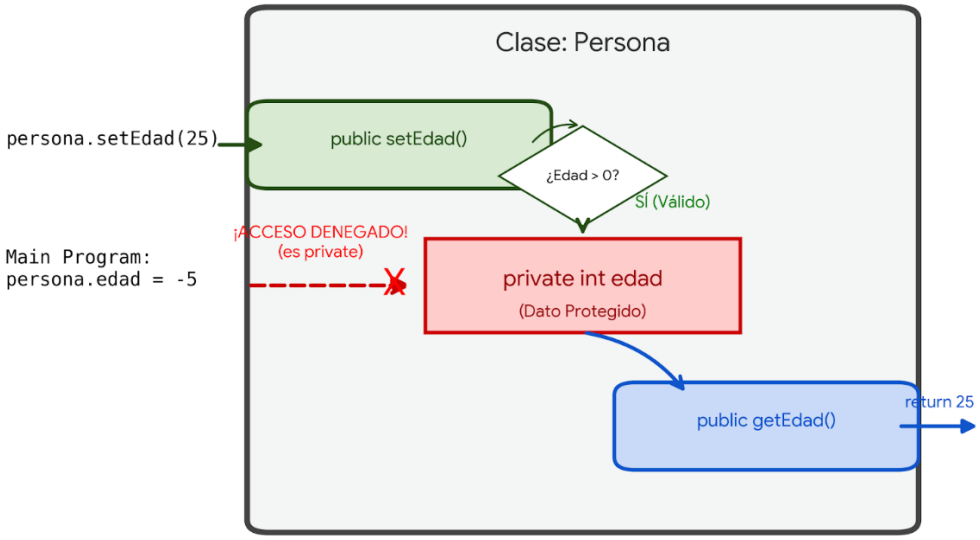
La ventaja del Setter es que nos permite **controlar** qué datos entran en nuestro objeto. Si hiciéramos el atributo público, no tendríamos control.

Ejemplo de validación (El Portero): Imagina una clase `Persona`. La edad no puede ser negativa.

Java
```
public class Persona {
    private int edad; // Privado: Nadie lo toca directamente

    // GETTER: Todo el mundo puede saber la edad
    public int getEdad() {
        return this.edad;
    }

    // SETTER: Solo aceptamos edades válidas
    public void setEdad(int nuevaEdad) {
        if (nuevaEdad >= 0 && nuevaEdad < 120) {
            this.edad = nuevaEdad;
            System.out.println("Edad actualizada correctamente.");
```

```
        } else {
            System.out.println("Error: La edad no es válida.");
            // Al no asignar nada, protegemos el dato original.
        }
    }
}
```

Si alguien intenta hacer `persona.setEdad(-5)`, el método lo detecta y rechaza el cambio, manteniendo la integridad del objeto.



Propiedades de Solo Lectura (Read-Only)

Los Getters y Setters nos permiten crear atributos que se pueden leer, pero no tocar.

- Si defines un **Getter** pero **NO defines un Setter**, el atributo será efectivamente de **"Solo Lectura"** para el mundo exterior.
- *Ejemplo:* Un atributo `numeroDeCuenta` que se asigna al nacer y nunca debe cambiar.

3.4 Inmutabilidad

Un objeto es **inmutable** cuando su estado interno no puede ser modificado una vez que el objeto ha sido creado. Nace con unos valores y muere con esos mismos valores.

Si necesitas cambiar la información de un objeto inmutable, no modificas el original; **creas uno nuevo** con los datos actualizados.

¿Cómo se crea una clase inmutable?

Para lograr este comportamiento en Java, debemos seguir una "receta" estricta que combina lo que hemos visto de encapsulamiento y constructores:

1. **No incluir Setters:** Eliminamos cualquier método que permita modificar atributos (`setNombre`, `cambiarPrecio`, etc.).
2. **Atributos `private`:** Para que nadie acceda directamente.

3. **Atributos `final`:** Usamos la palabra clave `final`. En Java, una variable `final` es una constante: solo se le puede asignar valor una vez. Si intentas cambiarla después, el código no compila.
4. **Inicialización completa en el Constructor:** Como no hay setters, la única oportunidad de dar valores es al nacer.

Ejemplo en Código

Imagina una clase que representa una coordenada GPS. Una vez fijada la coordenada, no debería cambiar "mágicamente".

Java
```
public class CoordenadaGPS {
    // 1. Atributos privados y FINAL (constantes una vez asignados)
    private final double latitud;
    private final double longitud;

    // 2. Constructor: La única vez que se asignan valores
    public CoordenadaGPS(double lat, double lon) {
        this.latitud = lat;
        this.longitud = lon;
    }

    // 3. Solo Getters (Lectura)
    public double getLatitud() {
        return latitud;
    }

    public double getLongitud() {
        return longitud;
    }

    // NO HAY SETTERS. No existe setLatitud().
}
```

El caso más famoso: La clase `String`

Probablemente ya has usado objetos inmutables sin saberlo. En Java, los **Strings son inmutables**.

Este es un error muy común de principiante:

Java
```
String texto = "hola";
texto.toUpperCase(); // Convertir a mayúsculas
System.out.println(texto); // ¿Qué imprime?
```

Imprime **"hola"** (en minúsculas).

- **¿Por qué?** Porque `toUpperCase()` no cambia el objeto `texto` original (no puede, es inmutable). Lo que hace es crear y **devolver un nuevo String** que es la versión en mayúsculas.

La forma correcta es:

Java
```
String texto = "hola";
String textoNuevo = texto.toUpperCase(); // Capturamos el nuevo
objeto
System.out.println(textoNuevo); // Imprime "HOLA"
```

Ventajas de la Inmutabilidad

¿Por qué limitarnos así?

1. **Seguridad en hilos (Thread Safety):** Si varios procesos intentan leer el objeto a la vez, no hay riesgo de que uno lo cambie mientras el otro lo lee. Son muy seguros.
2. **Simplicidad:** Sabes que si pasas un objeto a un método, ese método no te lo va a devolver "roto" o modificado.
3. **Uso en claves:** Son ideales para identificar cosas (como claves en mapas o bases de datos) porque su identidad es estable.

4 Relaciones básicas entre clases

En el diseño Orientado a Objetos, las clases rara vez trabajan solas. Establecen vínculos entre ellas para poder delegar tareas y compartir información.

La intensidad de estos vínculos varía: desde un contacto casual y momentáneo (Dependencia) hasta una relación de vida o muerte (Composición). Vamos a empezar por la relación más débil y común.

4.1 Dependencia (Uso)

La **Dependencia** es la relación más básica y temporal. Se conoce como la relación **"Usa-a"** (*Uses-a*).

Ocurre cuando una clase **A** necesita a una clase **B** para realizar una tarea específica, pero no la "posee" ni la guarda permanentemente. La relación empieza y termina con la ejecución de un método.

¿Cuándo sucede en Java?

Reconocemos una dependencia en el código cuando:

1. Una clase recibe un objeto de otra clase como **parámetro** en uno de sus métodos.
2. Una clase crea un objeto de otra clase **localmente** dentro de un método (y este muere al terminar el método).

Importante: En la dependencia, la clase B **NO** es un atributo de la clase A. No se guarda en la zona de variables de instancia. Es solo una herramienta de paso.

A continuación, vemos un ejemplo:

Imagina una clase `Carpintero` y una clase `Martillo`.

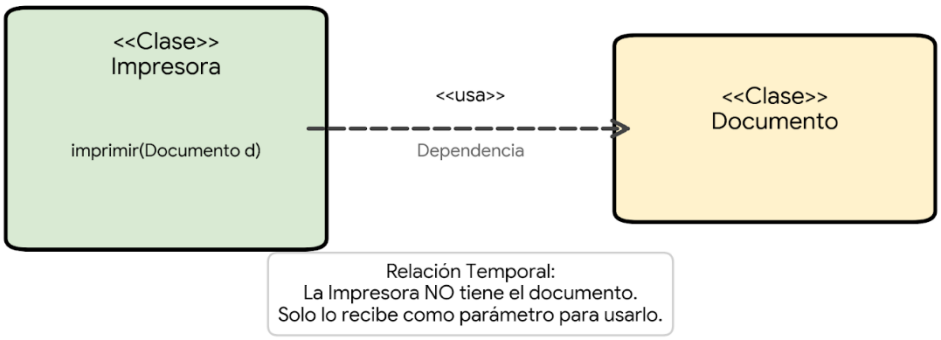
- El carpintero no nace con un martillo pegado a la mano (no es parte de su cuerpo).
- El carpintero no lleva el mismo martillo en el bolsillo las 24 horas del día.
- Sin embargo, para realizar la acción `clavar()`, el carpintero **necesita usar** un martillo momentáneamente.

Ejemplo en Código

Vamos a modelar una `Impresora` que imprime `Documentos`.

- La impresora no "tiene" el documento guardado dentro de sus circuitos para siempre.
- Solo lo necesita en el momento exacto de imprimir.

Java
```java
// Clase pasiva (el dato)
public class Documento {
    public String texto = "Hola mundo";
}

// Clase activa (quien usa)
public class Impresora {

    // NO hay atributos de tipo Documento aquí.
    // La impresora no "tiene" un documento.

    // RELACIÓN DE DEPENDENCIA:
    // El método recibe el documento, lo usa y se olvida de él.
    public void imprimir(Documento doc) {
        System.out.println("Imprimiendo: " + doc.texto);
    }
}
```



Si cambiamos la definición de la clase `Documento` (por ejemplo, borramos el atributo `texto`), la clase `Impresora` dejará de funcionar (error de compilación). Por eso decimos que la Impresora **depende** del Documento.

Representación en UML

En los diagramas de clases (Unified Modeling Language), la dependencia se dibuja con una **línea discontinua terminada en una flecha abierta** que apunta hacia la clase que es usada.

Características clave

1. **Temporalidad:** La relación es efímera.
2. **Acoplamiento débil:** Es la relación que menos "ata" a las clases entre sí (lo cual es bueno).
3. **Unidireccional:** La impresora sabe que existe el documento, pero el documento no tiene ni idea de que existe una impresora.

4.2 Asociación

La **Asociación** es una relación estructural que describe una conexión entre objetos. Ocurre cuando una clase "conoce" a otra y mantiene una referencia a ella durante un tiempo prolongado.

Se suele verbalizar como **"Tiene un..."** (aunque de forma más laxa que en la composición) o **"Conoce a..."**.

Diferencia clave con la Dependencia

Esta es la distinción que suelen preguntar en los exámenes:

- **En la Dependencia:** El objeto B aparece solo como **parámetro** de un método (`imprimir(Documento d)`). Cuando el método acaba, el objeto A se olvida de B.
- **En la Asociación:** El objeto B se guarda como un **atributo** de la clase A. El objeto A recuerda a B todo el tiempo.

Implementación en Java

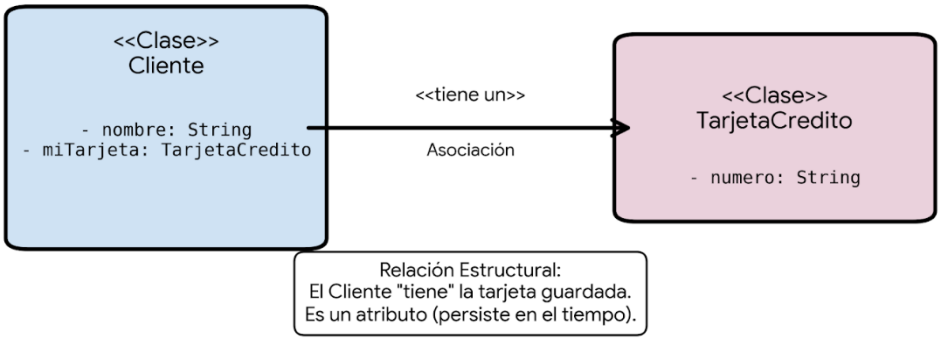
Para crear una asociación, declaramos una variable de instancia (atributo) cuyo tipo es la otra clase.

Ejemplo: Un Cliente y su TarjetaCredito

Un cliente no usa la tarjeta una vez y la tira; la lleva consigo (es su atributo).

Java
```java
public class TarjetaCredito {
    private String numero;
    // ... constructor y métodos
}

public class Cliente {
    private String nombre;
```

```
    // ASOCIACIÓN: El Cliente "tiene" una tarjeta guardada como
atributo.
    // Esta relación persiste en el tiempo.
    private TarjetaCredito miTarjeta;

    // Constructor: Vinculamos la tarjeta al nacer (o podría ser
con un setter)
    public Cliente(String nombre, TarjetaCredito tarjeta) {
        this.nombre = nombre;
        this.miTarjeta = tarjeta;
    }
}
```



Multiplicidad (Cardinalidad)

La asociación define cuántos objetos de un lado se relacionan con el otro.

1. **Uno a Uno (1:1):** Un `Cliente` tiene una `CuentaPrincipal`.
2. **Uno a Muchos (1:N):** Un `Profesor` tiene muchos `Estudiantes`.

    - *En código:* Esto se logra usando Arrays o Listas (que veremos más adelante), por ejemplo: `private Estudiante[] listaAlumnos;`.

Representación en UML

En los diagramas de clases, la asociación se representa con una **línea continua** que une ambas clases. A veces tiene una flecha abierta si la navegación es unidireccional (el Cliente conoce a la Tarjeta, pero la Tarjeta no sabe de quién es).

Resumen: Dependencia vs. Asociación

Característica	Dependencia	Asociación
Duración	Temporal (lo que dura el método)	Persistente (vida del objeto)

Característica	Dependencia	Asociación
Implementación	Parámetro de método o variable local	**Atributo de la clase**
Relación	"Usa a"	"Tiene a" / "Conoce a"
Línea UML	Discontinua (- - - >)	Continua (——>)

4.3 Agregación y Composición

A. Agregación (Relación Débil)

Es una relación de "Todo-Parte" donde la parte **puede existir independientemente** del todo. El "todo" agrupa a las partes, pero no es su dueño absoluto.

- **Ciclo de vida:** Si el objeto contenedor (el Todo) es destruido, los objetos contenidos (las Partes) **NO mueren**; siguen existiendo por su cuenta.
- **Analogía:** Un **Equipo de Fútbol** y sus **Jugadores**.
    - El equipo está formado por jugadores.
    - Si el equipo desaparece (se disuelve el club), los jugadores no mueren; quedan libres y pueden fichar por otro equipo.
- **En Código:** Normalmente, el objeto parte se crea fuera y se pasa al todo (por constructor o setter).

Java
```
public class Jugador { ... }

public class Equipo {
    private List<Jugador> jugadores; // Agregación

    public void ficharJugador(Jugador j) {
        // El jugador viene de fuera. Si borro el equipo, el
jugador sigue en memoria.
        this.jugadores.add(j);
    }
}
```

- **Símbolo UML:** Un **rombo vacío** (blanco) en el lado del "Todo".

B. Composición (Relación Fuerte)

Es una relación de "Todo-Parte" estricta. La parte **no tiene sentido ni existencia** sin el todo. El todo es responsable de crear y destruir a la parte.

- **Ciclo de vida:** Si el objeto contenedor muere, **arrastra consigo** a las partes, que también son destruidas.
- **Analogía:** Una **Casa** y sus **Habitaciones**.

- Una habitación es parte intrínseca de una casa.
- No puedes tener una "Habitación" flotando en la nada sin una casa.
- Si demueles la casa, la habitación deja de existir instantáneamente.

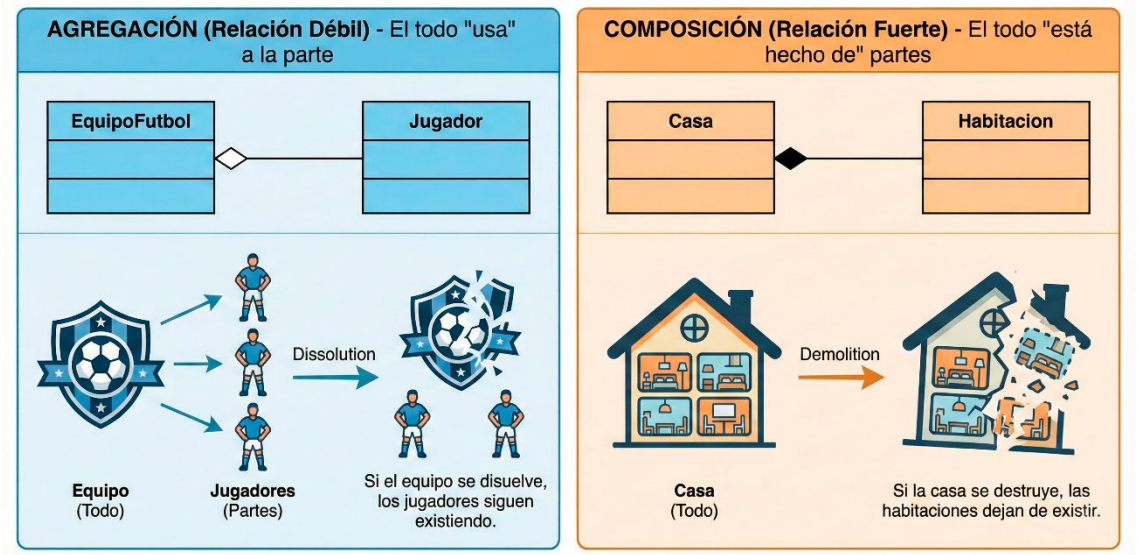
- **En Código:** El objeto parte se suele instanciar (`new`) **dentro** del constructor del todo. Nunca existe fuera.

Java
```
public class Habitacion { ... }

public class Casa {
    private Habitacion salon; // Composición

    public Casa() {
        // La Casa crea sus propias partes.
        // Si la Casa muere, nadie más tiene referencia al salón,
así que muere también.
        this.salon = new Habitacion();
    }
}
```

- **Símbolo UML:** Un **rombo relleno** (negro) en el lado del "Todo".

Tabla Comparativa

Característica	**Agregación (Débil)**	**Composición (Fuerte)**
Concepto	Agrupación / Colección	Estructura interna / Constituyente
Frase clave	"El todo **usa** a la parte"	"El todo **está hecho de** partes"
Independencia	La parte vive sola	La parte muere con el todo
Ejemplo	Ordenador y Ratón (puedes desenchufarlo)	Ordenador y CPU (sin CPU no hay ordenador)
UML	◇ (Rombo Blanco)	◆ (Rombo Negro)



4.4 Introducción a la Herencia (Generalización)

La **Herencia** es el mecanismo por el cual una clase (Hija) adquiere automáticamente todos los atributos y métodos de otra clase (Padre).

Se define como una relación **"Es-un"** (*Is-a*).

- Un Coche **es un** Vehículo.
- Un Gato **es un** Animal.
- Un Gerente **es un** Empleado.

¿Para qué sirve? (El principio DRY)

Su objetivo principal es la reutilización de código y evitar la repetición (Don't Repeat Yourself - DRY).

Si tienes 10 tipos de enemigos en un videojuego y todos tienen "vida" y "posición", no escribes ese código 10 veces. Creas una clase padre Enemigo con lo común, y los demás lo heredan gratis.

Terminología Clave

1. **Superclase (Padre / Base):** La clase general que contiene el código común (ej: `Vehiculo`).
2. **Subclase (Hija / Derivada):** La clase específica que hereda del padre y añade sus propios detalles (ej: `Coche`, `Moto`).

Sintaxis en Java: `extends`

En Java, la herencia se implementa con la palabra reservada **`extends`**.

Java
```
// 1. SUPERCLASE (Lo general)
class Vehiculo {
    String marca;
```

```
    void moverse() {
        System.out.println("El vehículo se mueve.");
    }
}

// 2. SUBCLASE (Lo específico)
// Al poner 'extends Vehiculo', Coche recibe AUTOMÁTICAMENTE
'marca' y 'moverse()'
class Coche extends Vehiculo {
    int numeroDePuertas; // Atributo exclusivo de Coche

    void tocarClaxon() { // Método exclusivo de Coche
        System.out.println("¡Mec mec!");
    }
}
```

¿Qué ocurre al usarlo? Cuando creas un objeto de la clase hija, tienes acceso a todo lo suyo más todo lo de su padre.

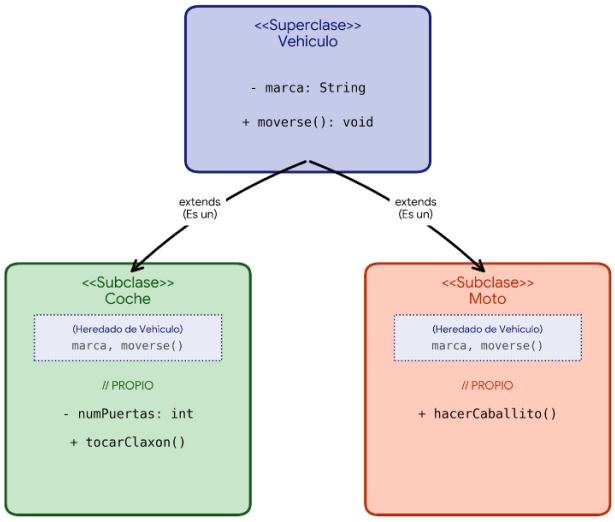
Java
```
Coche miCoche = new Coche();

miCoche.marca = "Toyota";     // ¡Heredado de Vehiculo!
miCoche.numeroDePuertas = 4; // Propio de Coche

miCoche.moverse();       // ¡Heredado de Vehiculo!
miCoche.tocarClaxon();   // Propio de Coche
```

Diferencia vital: Asociación vs. Herencia

Es un error muy común de principiante confundirlas. Hazte siempre la **"Prueba de la frase"**:

- **¿Un Coche "tiene un" Motor?** SÍ -> Esto es **Composición/Asociación**.
- **¿Un Coche "es un" Motor?** NO -> No uses herencia aquí.
- **¿Un Coche "es un" Vehículo?** SÍ -> Esto es **Herencia**.

Más info: En Java, una clase solo puede heredar de **una sola** clase padre (Herencia simple). No puedes hacer `extends Vehiculo, Maquina`.



5 Principios básicos de la orientación a objetos

Los cuatro pilares de la POO son: **Abstracción, Encapsulamiento, Herencia y Polimorfismo**. Vamos a desglosar el primero, que es el punto de partida de todo diseño.

5.1 Abstracción: Simplificación de la realidad

La **Abstracción** es el proceso mental de ignorar los detalles irrelevantes de un objeto para centrarnos únicamente en las características que son esenciales para resolver nuestro problema específico.

En términos sencillos: **Abstraer es simplificar.**

La Analogía del Mapa

La frase clásica para definir la abstracción es: *"El mapa no es el territorio"*.

- Un mapa de carreteras de España es una abstracción de España.
- No tiene árboles, ni gatos, ni el color de las casas. Solo tiene carreteras y ciudades.
- **¿Por qué?** Porque si estás conduciendo, el color de las casas es **irrelevante**. Incluirlo solo añadiría "ruido" y complejidad inútil.

La Abstracción depende del Contexto

Lo que es esencial y lo que es irrelevante depende totalmente de **qué programa estás creando**. Una misma entidad del mundo real puede tener abstracciones (Clases) totalmente diferentes.

Ejemplo: La entidad "Persona"

1. **Contexto A: Una App de Citas (Tinder)**
   - *Datos relevantes (Abstracción):* `foto`, `hobbies`, `edad`, `descripción`.
   - *Irrelevante (Se ignora):* Su grupo sanguíneo, su número de la seguridad social, su historial médico.
2. **Contexto B: Un Sistema de Hospital**
   - *Datos relevantes (Abstracción):* `grupoSanguineo`, `alergias`, `historialMedico`, `peso`.
   - *Irrelevante (Se ignora):* Sus hobbies, su música favorita o si busca pareja.

La Clase como herramienta de Abstracción

En Java, la Clase es la materialización de este principio.

Cuando escribes class Coche { ... }, estás tomando una decisión de diseño: estás decidiendo qué atributos y métodos "merecen" entrar en tu software y cuáles se quedan fuera.

- **Mal diseño:** Intentar modelar el objeto real al 100% (ponerle al Coche `marcaDeLosTornillos`, `tipoDeTelaDelAsiento`, etc., cuando solo estás haciendo una app de multas).
- **Buen diseño (Buena abstracción):** Capturar solo los datos necesarios para que el negocio funcione.

La abstracción nos permite manejar la complejidad. Si tuviéramos que programar el mundo con todo detalle, nunca terminaríamos. Nos centramos en el "qué es" y "qué hace" en nuestro contexto, ignorando el resto.

5.2 Encapsulamiento

El **Encapsulamiento** es el principio que consiste en empaquetar los datos (atributos) y los métodos (comportamiento) dentro de una misma unidad (la Clase) y restringir el acceso directo a algunos de los componentes del objeto.

Su nombre viene de "cápsula": creamos una cáscara protectora alrededor de nuestro código.

Este principio tiene dos objetivos fundamentales que resumen lo que hemos visto en la unidad:

1. Agrupación (Cohesión)

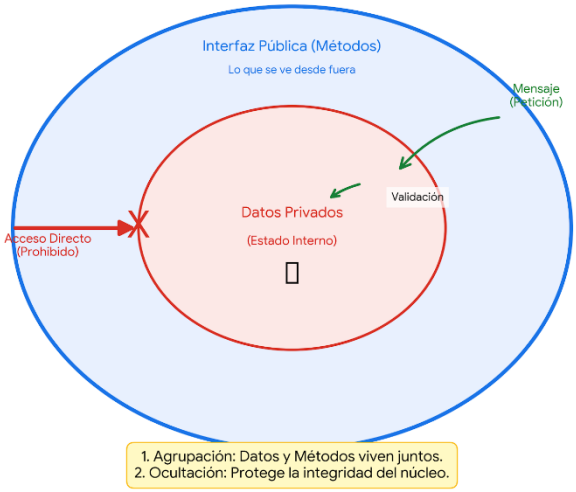
Antes de la POO, los datos viajaban por un lado y las funciones por otro. El encapsulamiento dice: *"Los datos y las funciones que manipulan esos datos deben vivir juntos"*.

- Si tienes datos de un `Cliente`, los métodos para validar el DNI o cambiar la dirección deben estar **dentro** de la clase `Cliente`, no esparcidos por el programa.

2. Ocultación de Información (Data Hiding)

Es la capacidad de prohibir el acceso a la estructura interna del objeto.

- **El Qué vs. El Cómo:** Al mundo exterior le ofrecemos una **Interfaz Pública** (qué puedes hacer: `arrancar()`, `frenar()`) y ocultamos la **Implementación Privada** (cómo lo haces: inyección de combustible, pistones, bujías).
- Esto se logra, como vimos en el punto 2.3, usando los modificadores `private` y exponiendo solo métodos `public` controlados (Getters/Setters).



Beneficios clave del Encapsulamiento

¿Por qué es un pilar fundamental?

1. **Integridad de los datos:** Evita que el objeto llegue a un estado inválido. Un objeto es el único responsable de su propio estado. Nadie puede ponerle una "edad negativa" si el objeto no lo permite en sus métodos.
2. **Mantenibilidad (Bajo Acoplamiento):** Puedes cambiar el código interno de la clase sin romper el código de quien la usa.

    - *Ejemplo:* Si hoy guardas la fecha como `String` y mañana decides cambiarla a un objeto `Date`, si tienes el dato encapsulado, solo cambias tu clase. Si el dato fuera público, tendrías que cambiar todo el programa que accediera a esa variable.

El encapsulamiento es el mecanismo de **seguridad y defensa** del objeto.

5.3 Herencia: Reutilizar código

La **Herencia** es el mecanismo fundamental de la POO que permite definir una nueva clase basándose en una clase existente. La nueva clase adquiere (hereda) automáticamente todos los atributos y métodos de la clase original, pudiendo añadir sus propias características o modificar las heredadas.

Su propósito principal es cumplir con el principio **DRY (Don't Repeat Yourself)**: si ya has programado algo una vez, no vuelvas a escribirlo; herédalo.

La relación "Es-un" (Is-a)

Para usar la herencia correctamente, siempre debemos validar la relación lógica con la prueba "Es un...".

- Un `Gato` **es un** `Animal`. (Correcto: Herencia)
- Un `Director` **es un** `Empleado`. (Correcto: Herencia)
- Un `Coche` **es una** `Rueda`. (Incorrecto: Un coche *tiene* ruedas, pero no *es* una rueda. Eso sería composición).

Terminología. En esta relación jerárquica distinguimos dos roles:

1. **Superclase (Clase Padre / Base):** Es la clase general que contiene el código común a todos. (Ej: `Vehiculo`).
2. **Subclase (Clase Hija / Derivada):** Es la clase específica que hereda del padre y se especializa. (Ej: `Coche`, `Moto`).

Sintaxis en Java: `extends`

En Java, utilizamos la palabra reservada **`extends`** para indicar que una clase hereda de otra.

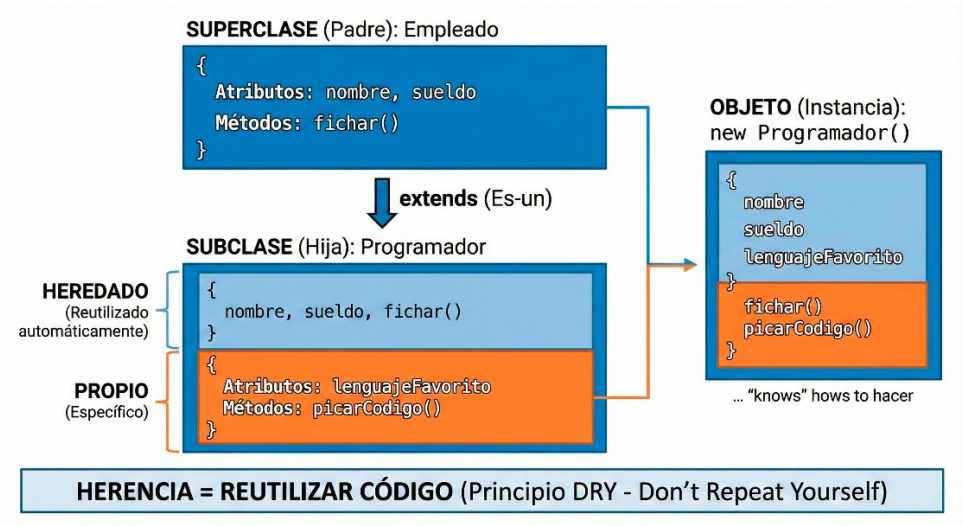
Ejemplo Práctico: Imagina que estamos programando un sistema para una empresa. Todos los trabajadores tienen nombre y sueldo. Sin herencia, tendríamos que repetir ese código en la clase Programador, en Gerente, en Becario, etc.

Con herencia, lo hacemos así:

Java
```
// 1. SUPERCLASE: Define lo común
// Todos los empleados del mundo tienen nombre y fichan a la
entrada.
public class Empleado {
    String nombre;
    double sueldo;

    public void fichar() {
        System.out.println("El empleado " + nombre + " ha fichado a
las 8:00");
    }
}

// 2. SUBCLASE: Se especializa
// Al poner 'extends Empleado', el Programador YA TIENE nombre y
método fichar().
// No hace falta escribirlos de nuevo.
public class Programador extends Empleado {

    // Atributo exclusivo de Programador
    String lenguajeFavorito;

    // Método exclusivo de Programador
    public void picarCodigo() {
        System.out.println("Estoy programando en " +
lenguajeFavorito);
    }
}
```
¿Qué ocurre al usarlo?

Cuando creas un objeto `Programador`, este nace "sabiendo" hacer todo lo que hace un `Empleado` más lo suyo propio.
Java
```
Programador junior = new Programador();
junior.nombre = "Ana";          // Heredado de Empleado
junior.lenguajeFavorito = "Java"; // Propio de Programador

junior.fichar();      // Ejecuta el código definido en Empleado
junior.picarCodigo(); // Ejecuta el código definido en Programador
```

Importante: Java soporta Herencia Simple, lo que significa que una clase solo puede tener un único padre (`extends A`). No puedes hacer `extends A, B`. Esto se hace para evitar problemas de ambigüedad (como el "Problema del Diamante").



5.4 Polimorfismo: Flexibilidad de comportamiento

La palabra **Polimorfismo** viene del griego: Poly (muchas) y Morphe (formas). En programación, se define como **la capacidad de un objeto de adoptar muchas formas**.

Técnicamente, permite que una referencia de una **Superclase** (Padre) pueda apuntar a un objeto de cualquiera de sus **Subclases** (Hijas) y actuar correctamente según el tipo real del objeto.

1. El concepto clave: Una orden, distintas respuestas

Imagina que eres un director de orquesta. Tienes un violinista, un pianista y un batería.

- Tú das una única orden genérica: **"¡Tocad!"**.
- No necesitas ir uno por uno diciendo "Tú rasga la cuerda", "Tú golpea la tecla", "Tú pega al tambor".
- Tú dices "Tocad" (el mensaje es el mismo), y cada músico interpreta esa orden a su manera.

Eso es polimorfismo: enviar el mismo mensaje a objetos diferentes y que cada uno responda según su propia naturaleza.

2. Requisito técnico: La Sobreescritura (Overriding)

Para que esto funcione, necesitamos que las clases hijas modifiquen el comportamiento heredado del padre.

Esto se llama Sobreescritura de métodos.

Usamos la anotación `@Override` para indicar que estamos cambiando la lógica del padre.
Java
```
// CLASE PADRE
class Animal {
    void hacerSonido() {
```

```
        System.out.println("El animal hace un ruido genérico");
    }
}

// CLASE HIJA 1
class Perro extends Animal {
    @Override
    void hacerSonido() {
        System.out.println("¡Guau!");
    }
}

// CLASE HIJA 2
class Gato extends Animal {
    @Override
    void hacerSonido() {
        System.out.println("¡Miau!");
    }
}
```

3. La Magia: Referencias Polimórficas

Aquí es donde cambia todo lo que sabías sobre variables. Hasta ahora, si querías un Perro, usabas una variable tipo `Perro`. Con polimorfismo, podemos usar una variable tipo `Animal`.
Java

```
// Referencia del Padre (Animal) -> Objeto del Hijo (Perro)
Animal miMascota = new Perro();

// ¿Qué pasa si llamo al método?
miMascota.hacerSonido();
```

¿Qué imprime esto? Aunque la variable es de tipo Animal, Java es lo suficientemente listo para ver que el objeto real en memoria es un Perro.

- Resultado: **"¡Guau!"** (Ejecuta la versión del hijo).

Esto se llama **Vinculación Dinámica** (Dynamic Binding): Java decide en tiempo de ejecución qué método usar.

4. ¿Para qué sirve esto? (La utilidad real)

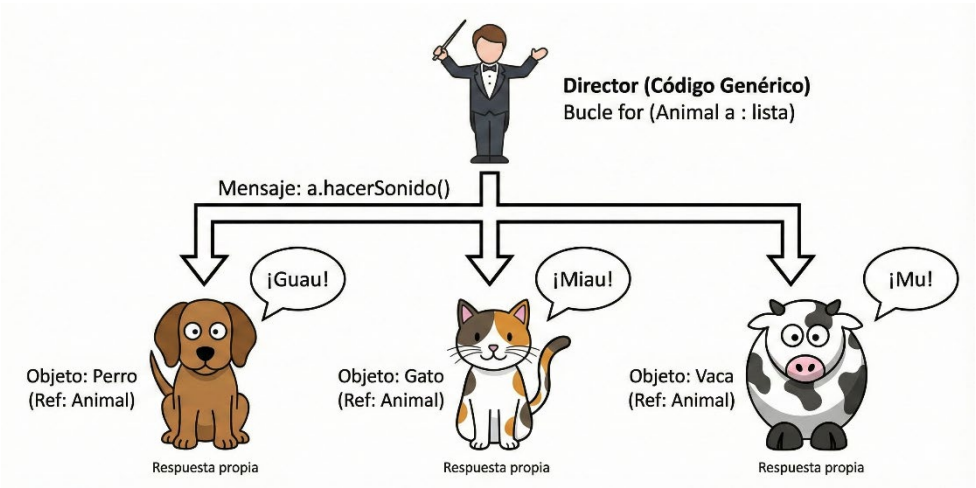
Imagina que tienes un videojuego con 50 tipos de monstruos distintos. Sin polimorfismo, necesitarías 50 listas diferentes (`List<Orco>`, `List<Dragon>`, etc.).

Con polimorfismo, puedes tratarlos a todos como `Monstruo` de forma uniforme:
Java

```
// Array polimórfico: Puedo mezclar Perros y Gatos porque ambos SON
Animales
Animal[] listaDeAnimales = new Animal[2];
listaDeAnimales[0] = new Perro();
listaDeAnimales[1] = new Gato();

// Bucle genérico
for (Animal a : listaDeAnimales) {
```

```
    // Aquí está la magia:
    // No sé si 'a' es perro o gato, ni me importa.
    // Solo le digo "haz sonido" y cada uno lo hace a su manera.
    a.hacerSonido();
}
// Salida:
// ¡Guau!
// ¡Miau!
```

Si mañana inventas un nuevo animal (`Vaca`), no tienes que tocar el código del bucle `for`. Solo creas la clase `Vaca` y funciona automáticamente. El código es **extensible**.



Resumen

Esta unidad marca el paso de la programación secuencial a la **Arquitectura de Software**. Ya no escribimos instrucciones sueltas; modelamos entidades que interactúan entre sí.

1. El Núcleo: Clase vs. Objeto

La distinción fundamental. Sin esto claro, nada funciona.

Concepto	Definición	Analogía	En código
Clase	Es la plantilla, el molde o la definición abstracta. No ocupa memoria de ejecución, solo define la estructura.	El **Plano** de la casa.	`public class Coche { ... }`
Objeto	Es la instancia concreta creada a partir de la clase. Tiene vida en la memoria RAM.	La **Casa** real construida.	`new Coche();`

Anatomía de una Clase

Se compone de dos miembros principales:

1. **Atributos (Estado):** Variables que guardan los datos (`String color;`, `int velocidad;`).
2. **Métodos (Comportamiento):** Funciones que definen qué puede hacer el objeto (`arrancar()`, `frenar()`).

El Constructor

- **Misión:** Inicializar el objeto para que nazca con datos válidos.
- **Reglas:** Se llama **igual** que la clase y **no tiene tipo de retorno**.
- **`this`:** Palabra clave para diferenciar los atributos de la clase de los parámetros del método.

2. Gestión de Memoria: Referencias

En Java, las variables de objetos no guardan el objeto, guardan la **dirección** para encontrarlo.

- **Stack (Pila):** Memoria rápida donde vive la variable (el "mando a distancia").
- **Heap (Montículo):** Memoria grande donde vive el objeto real.
- **Consecuencia:** Si haces `Coche a = b`, no copias el coche, copias el mando. Ambos apuntan al mismo objeto. Si uno lo modifica, el otro ve el cambio.
- **`null`:** Una referencia que no apunta a ningún sitio. Si intentas usarla -> `NullPointerException`.

3. Encapsulamiento (Seguridad)

El principio de "Caja Negra". Ocultar los detalles internos y mostrar solo lo necesario.

Modificadores de Acceso (De más cerrado a más abierto)

1. **`private`:** Solo visible dentro de la **misma clase**. (Obligatorio para atributos).
2. **(default):** Visible en el mismo **paquete**.
3. **`protected`:** Visible en paquete y **subclases** (hijas).
4. **`public`:** Visible para **todo el mundo**. (Usado para métodos).

El Patrón Getter/Setter

- **Atributos `private`:** Nadie los toca directamente.
- **Métodos `public`:**
    - **Getter:** Para leer el valor (`getEdad()`).

- **Setter:** Para modificar el valor con validación (`setEdad(int e)`). *Si la edad es negativa, no la guardo.*

4. Relaciones entre Clases

Los objetos no viven aislados. La intensidad de su relación define la arquitectura.

Tipo	Definición	Frase Clave	Ejemplo
Dependencia	Relación temporal. Una clase usa a otra en un método y luego la olvida.	*"Usa un..."*	Impresora usa Documento.
Asociación	Relación estructural. Una clase tiene a otra como atributo permanentemente.	*"Tiene un..."*	Cliente tiene Tarjeta.
Agregación	Tipo de asociación. "Todo-Parte" débil. Si el todo muere, la parte sobrevive.	*"Agrupa a..."*	Equipo y Jugadores.
Composición	Tipo de asociación. "Todo-Parte" fuerte. Si el todo muere, la parte muere.	*"Compuesto de..."*	Casa y Habitaciones.
Herencia	Relación de jerarquía y especialización. Reutilización de código.	*"Es un..."*	Coche es un Vehículo.

5. Los 4 Pilares de la POO (Teoría)

1. **Abstracción:** Simplificar la realidad. Elegir solo los atributos y métodos relevantes para el problema actual, ignorando los detalles innecesarios.
2. **Encapsulamiento:** Proteger los datos dentro del objeto y controlar el acceso mediante una interfaz pública.
3. **Herencia:** Mecanismo (`extends`) para crear nuevas clases basadas en otras existentes, heredando su comportamiento para no repetir código (DRY).
4. **Polimorfismo:** Capacidad de objetos de distintas clases (pero con un padre común) de responder al mismo mensaje de formas diferentes.
   - *Clave:* Sobreescritura de métodos (`@Override`) y uso de referencias del tipo padre (`Animal a = new Perro()`).

Ejemplo de Sintaxis Rápida

Java
```
// Definición de Clase (Abstracción y Encapsulamiento)
public class Coche extends Vehiculo { // Herencia
```

```java
    // Atributos Privados (Estado)
    private String marca;

    // Constructor (Inicialización)
    public Coche(String marca) {
        this.marca = marca;
    }

    // Método Público (Comportamiento)
    public void acelerar() {
       // Lógica...
    }

    // Sobreescritura (Polimorfismo)
    @Override
    public void moverse() {
        System.out.println("El coche rueda");
    }
}

// Uso (Instanciación)
Coche miCoche = new Coche("Ford"); // 'new' reserva memoria
```